UNIVERSITY OF Southampton
School of Electronics and Computer Science

Comp1202: Programming I

Variables - Primitives, Objects and Scope

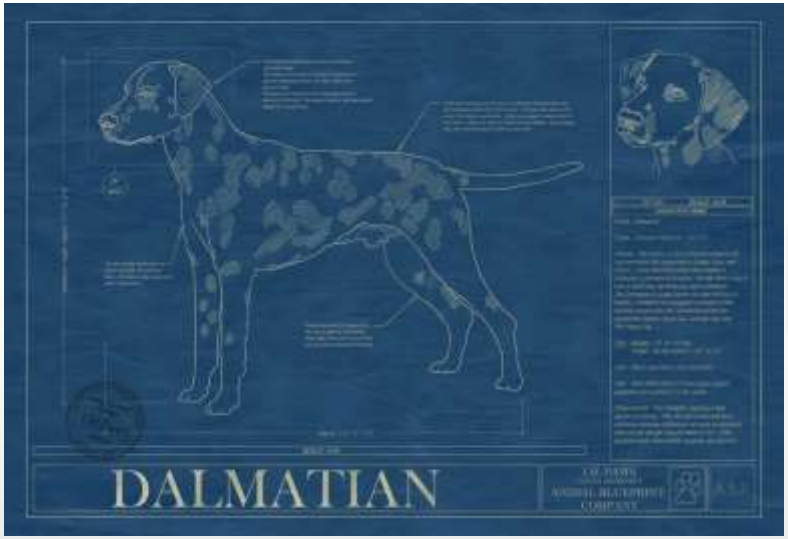
Coming up

- Quick Recap:
  - Objects vs. Classes
  - Conditional statements
- Variables: Primitives and Objects
  - How they are Defined
  - How they are Stored
  - How they are Passed
- Introduction to Scope

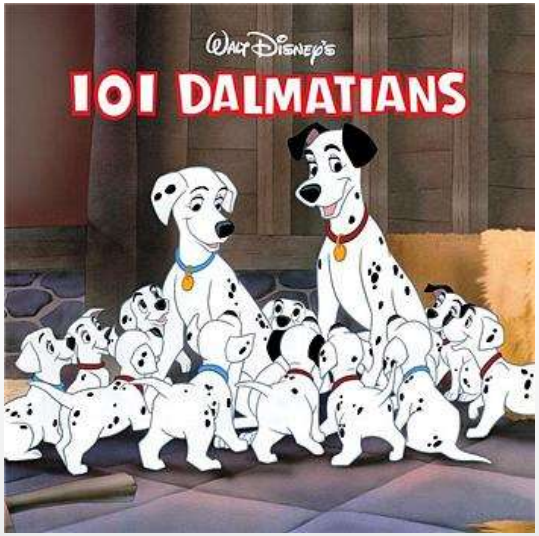
Recap

Classes and Objects

1 Class

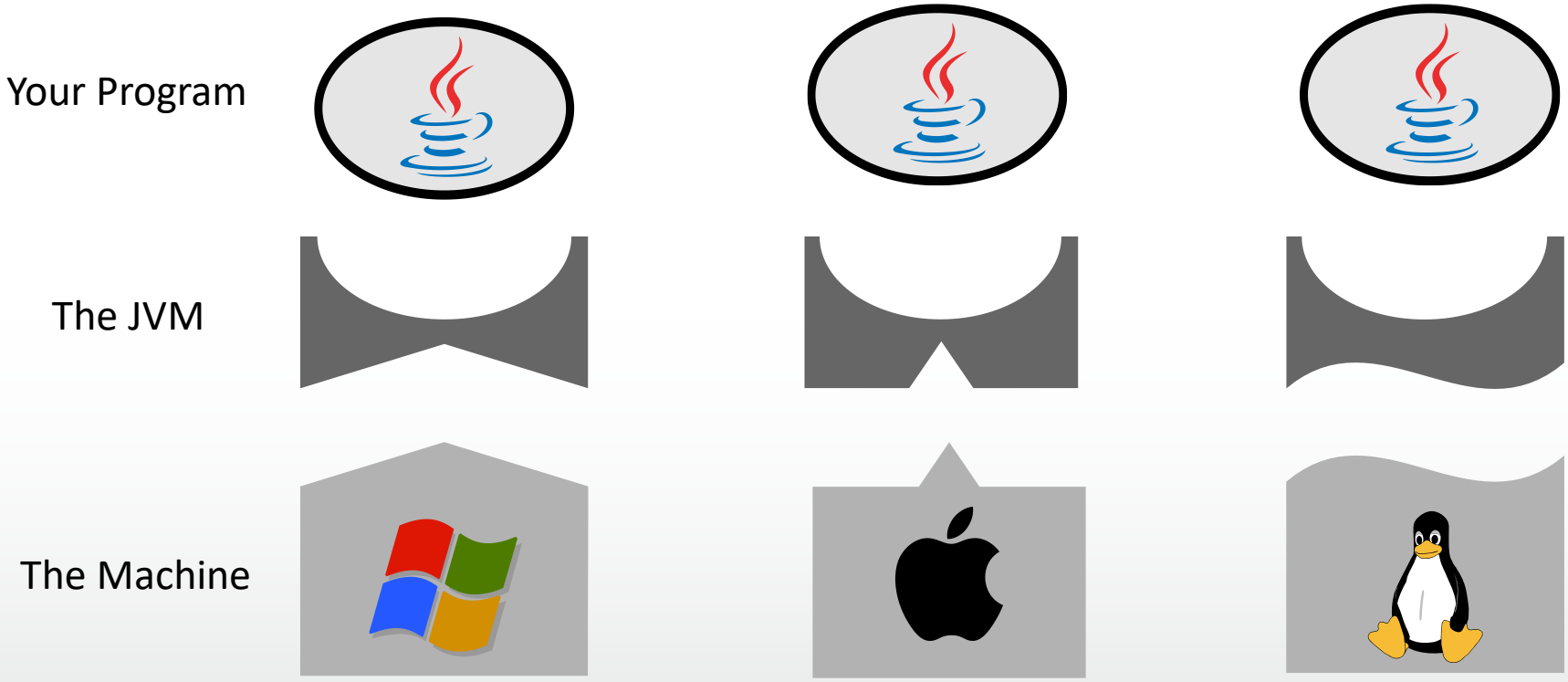


101 Objects



http://www.animalblueprintcompany.com/

The JVM?



So the JVM is software that allows your Java program to run on any platform where there is a JVM

Structure

Source file

Dog.java

Class

Member Variable

Method

Statement

```
public class Dog
{
    public String noise = "Woof";

    public void bark()
    {
        System.out.println(noise);
    }
}
```

The main method

- Creates the objects you need and tells them what to do, a bit like a conductor in an orchestra
- It doesn't matter which class you put the main method in. You can put it in its own one if you like
  - But there must be 1 and only 1 in the whole program (program is a set of classes that work together)

Dog.java

```
public class Dog
{
   public String noise = "Woof";

   public void bark() {
        System.out.println(noise);
   }

   public static void main(String[] args){
         Dog d = new Dog();
         d.bark();
     }
}
```

if/else

```
public class Account{

    int balance;                    //the bank balance
    boolean active = true;          // true if the account is active

    //some code omitted

    public void withdrawFiver{
        if(!active){
            System.out.println("Your account isn't active");
            System.out.println("No withdrawal allowed");
        }else {
            balance = balance - 5;
        }
    }
}
```

Variables: Primitives and Objects

Variables

We know that variables store ‘things’ like ints, booleans and Elephants

- Variables that are part of a class are called **member variables**
  - Sometimes called properties or fields

- Variables can store a **primitive type**
- Variables can store an **object reference**

	Primitive Type	**Object Reference**
Defined?		
Stored?		
Passed?		

Primitive Types are used for simple pieces of Data

Object References are used for handling Objects

They differ in how they are **defined**, **stored** and **passed**

	Primitives	Objects
Defined?	defined in Java	
Stored?		
Passed?		

- Primitive types are for simple data, e.g:
  - int whole numbers
  - float floating point numbers (i.e. decimals)
  - char a single character
  - boolean true or false

- They are an integral part of the Java Language. We just use them

	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?		
Passed?		

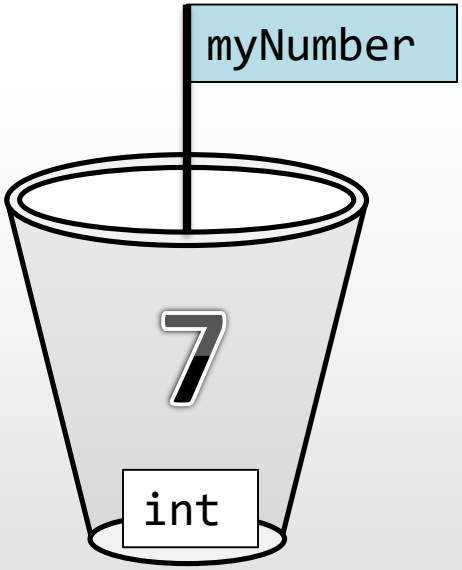
- Objects can be of a type defined by:
  - a class you write
  - a class someone else writes
    - Java includes a library of useful classes, ones that can read files on your computer or output sounds and many other tasks. We use these a lot later on.

	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?	stored directly in variables	
Passed?		

- A variable is a space in memory that the computer uses to store a value

- It's like a cup

```
int myNumber;
myNumber = 7;
```



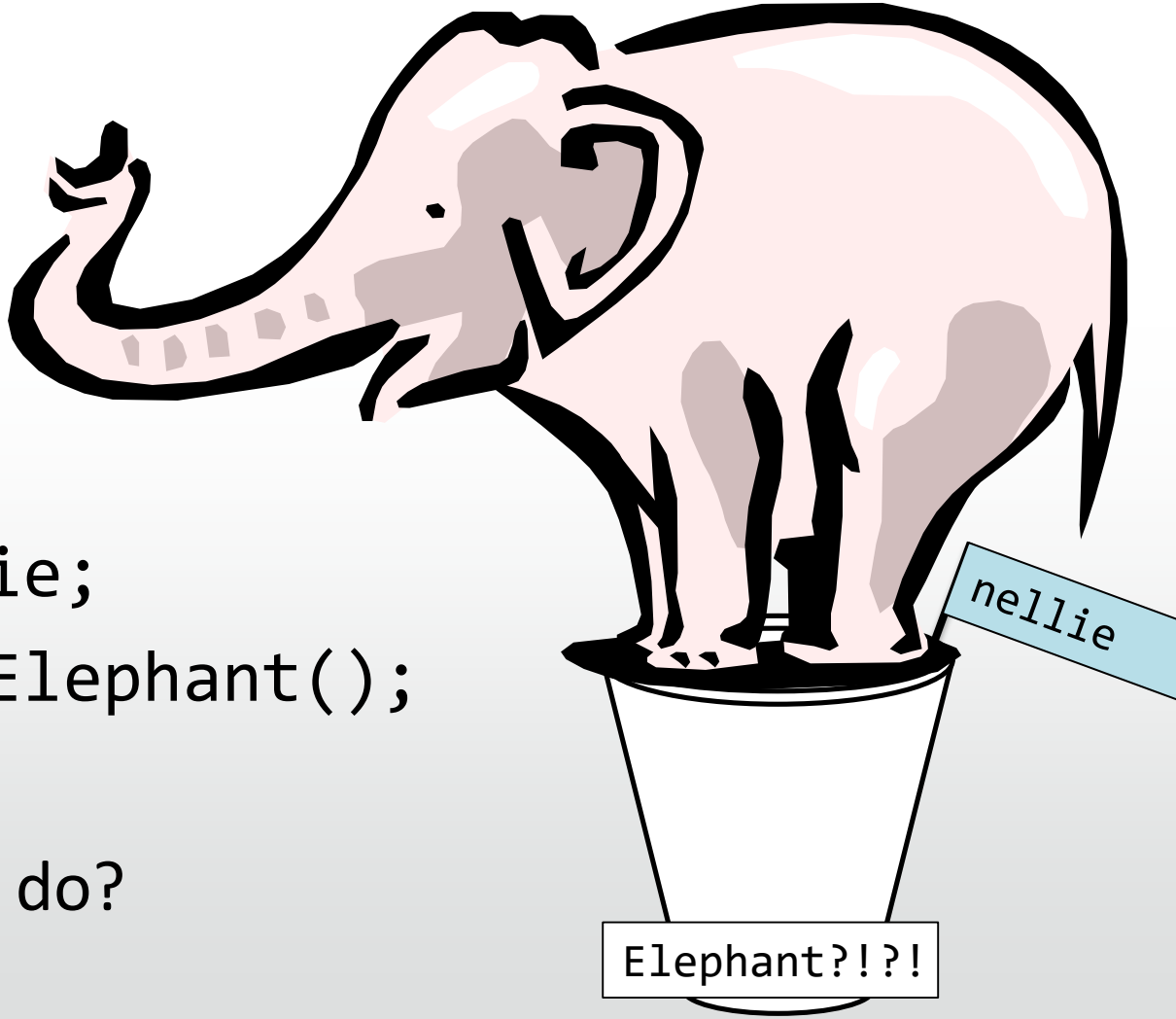
- **A primitive 'fits into' a cup**

	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?	stored directly in variables	a reference is stored in the variable
Passed?		

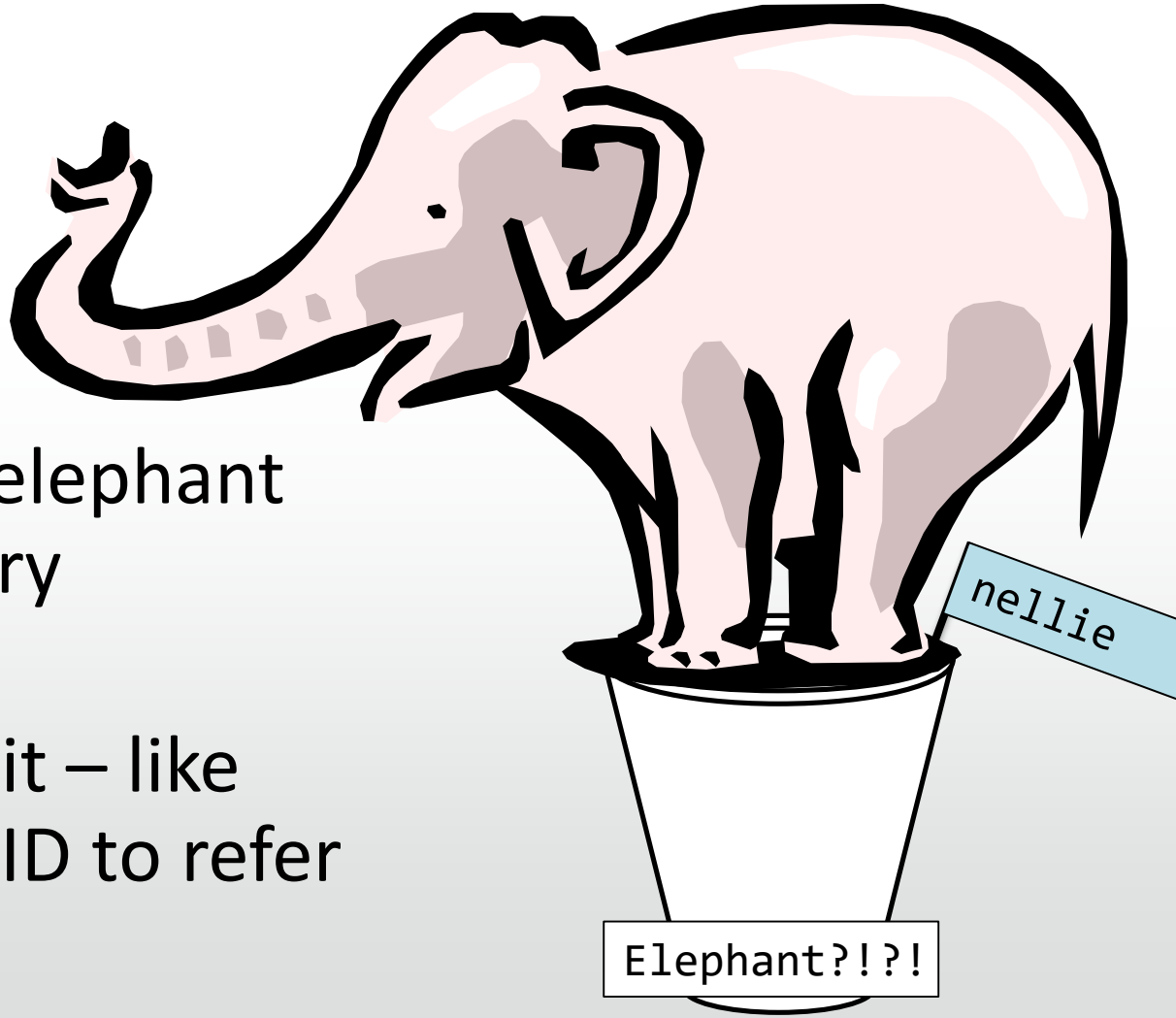
- An object does not fit into a cup...

```
Elephant nellie;
nellie = new Elephant();
```

- So what does Java do?

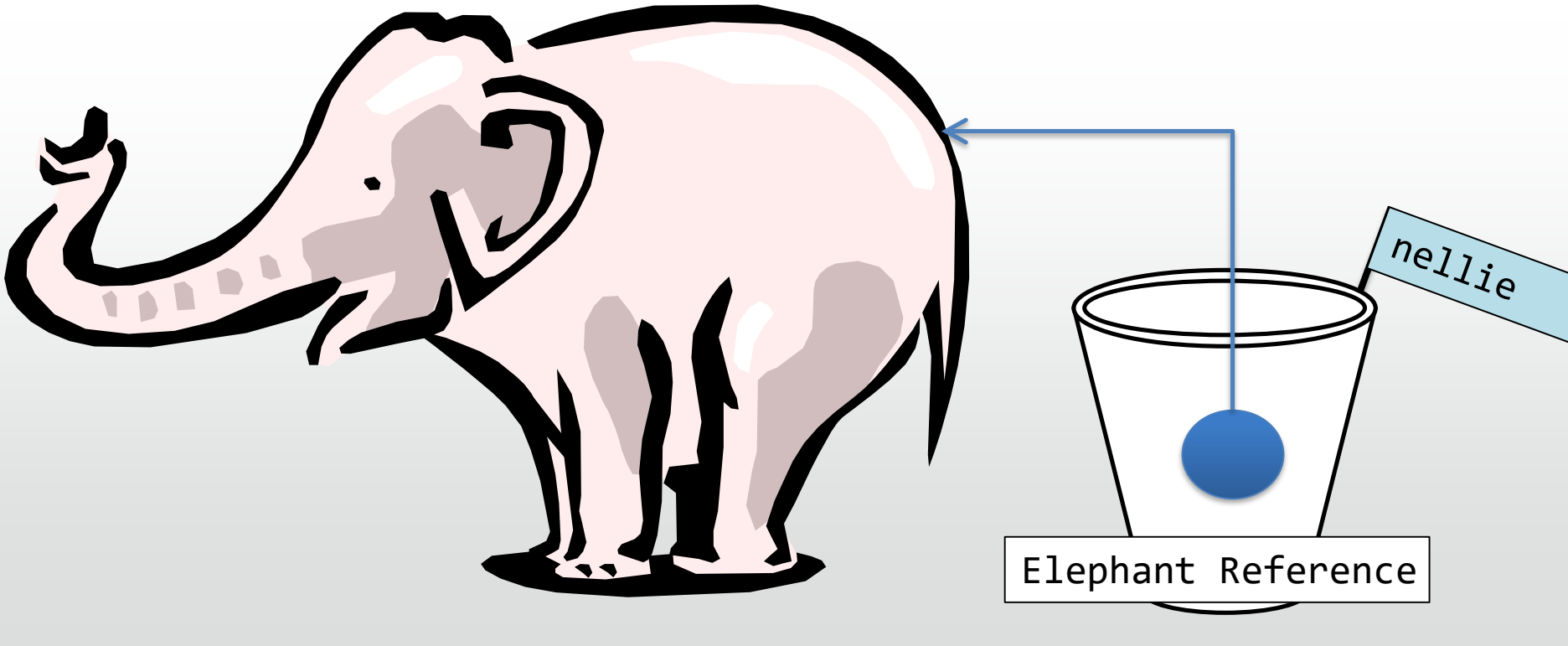


	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?	stored directly in variables	a reference is stored in the variable
Passed?		



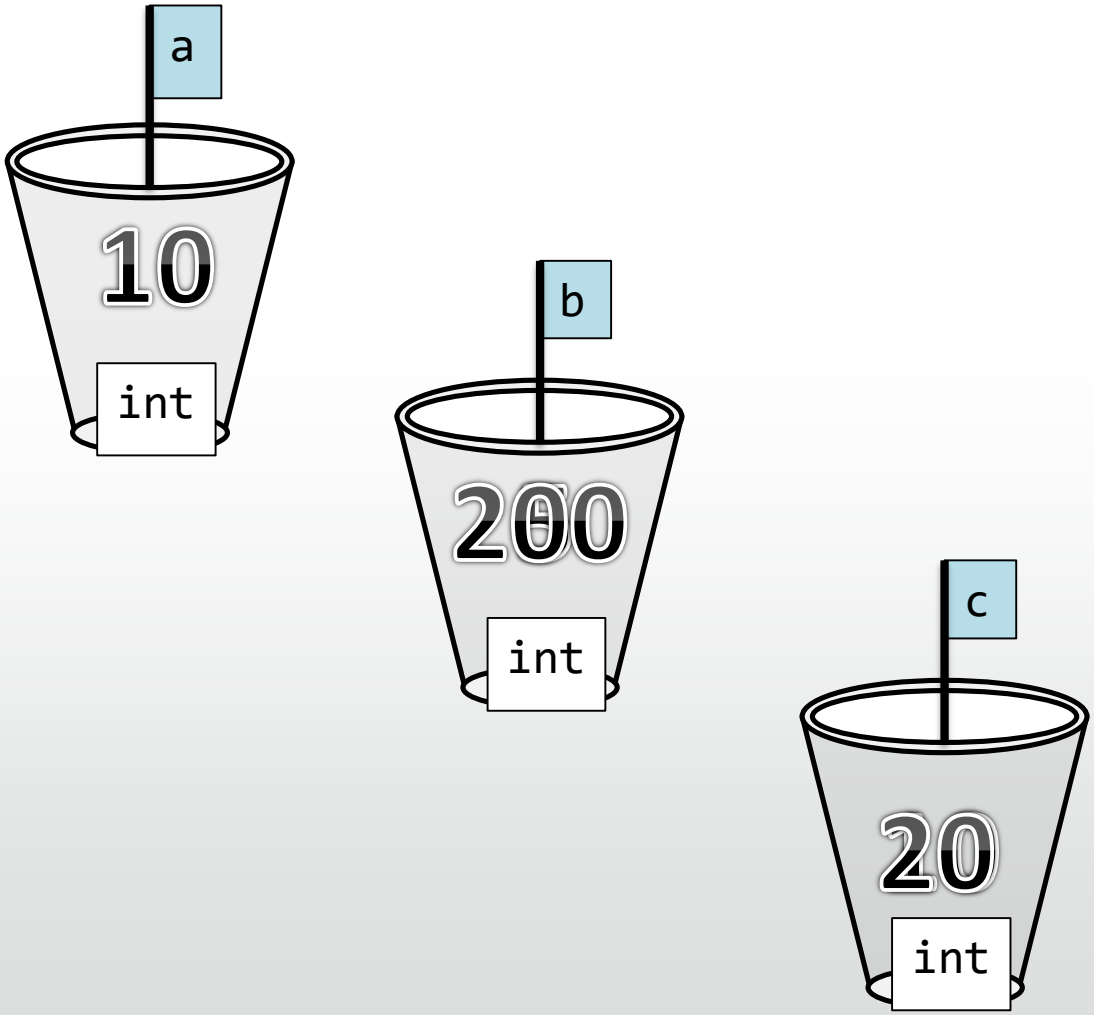
- Java leaves the elephant object in memory somewhere...
- And **references** it – like using a student ID to refer to a student

	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?	stored directly in variables	a reference is stored in the variable
Passed?		



	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?	stored directly in variables	a reference is stored in the variable
Passed?	Pass by copy	

```
int a;
a = 10;
int b;
b = 5;
int c;
c = a;
c = 2*c;
b = a*c;
```

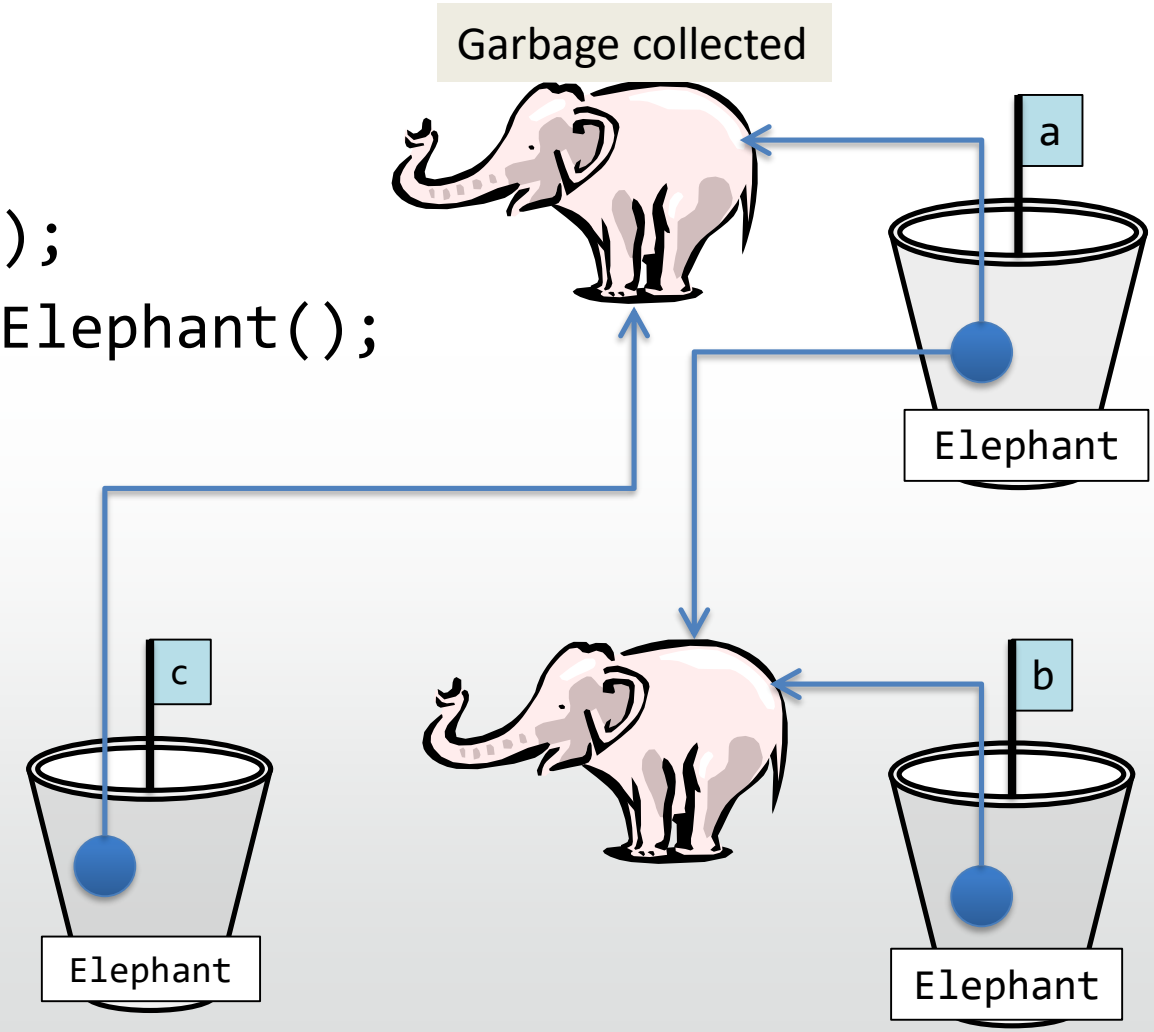


	Primitives	Objects
Defined?	defined in Java	defined in Classes
Stored?	stored directly in variables	a reference is stored in the variable
Passed?	Pass by copy	Pass by reference

```
Elephant a;
a = new Elephant();
Elephant b = new Elephant();

Elephant c;
c = a;
a = b;

c = null;
```



Can Elephants Change Their Names

```
public static void main(String[] args) {
        Elephant a;
        a = new Elephant();
        a.name = "Elephant1";
        Elephant b;
        b = a;
        b.name = "Elephant2";
        // What is the name of the elephants now?
        System.out.println("a is "+a.name+", b is "+b.name);
}
```

Introduction to Scope

A word about Local variables

- **Member variables** (object properties) are one sort of variable.
  - They store values through the life of an object.
  - They are accessible in any method in the class.

- Methods can include shorter-lived **local variables**.
  - They exist only as long as the method is being executed.
  - They are only accessible from within the method.
  - They are valid only after their declaration.

- This is called scope.

The rule of thumb...

- ...is that a variable can be seen anywhere within the {} that it was **declared** in

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

}
```

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

}
```

Balance is a member variable, and is visible in every method of the class

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

   public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
   }

}
```

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

   public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
   }

}
```

Tenner is a local variable, it only exists for the duration of the method in which it is declared.

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

   public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
   }

   public void withdrawFifty(){
        balance = balance – (tenner * 5);
   }

}
```

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

   public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
   }

   public void withdrawFifty(){
        balance = balance – (tenner * 5);
   }

}
```

Because tenner is a local variable, it cannot be seen in any other methods – so the withdrawFifty method shown here will not compile

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

   public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
   }

   public void withdrawFifty(){
        balance = balance – (tenner * 5);
   }

   public void closeAccount(){
        int balance = 0;
   }
}
```

```
public class Account{

   int balance = 100;

   public void withdrawFiver(){
        balance = balance -5;
   }

   public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
   }

   public void withdrawFifty(){
        balance = balance – (tenner * 5);
   }

   public void closeAccount(){
        int balance = 0;
   }
}
```

Here we get a conflict because balance has been declared twice, in the class as a member variable and in the closeAccount method as a local variable.

Any modifications to balance in closeAccount will default to the local version, and the member variable will remain unchanged

```
public class Account{

    int balance = 100;

    public void withdrawFiver(){
        balance = balance -5;
    }

    public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
    }

    public void withdrawFifty(){
        balance = balance – (tenner * 5);
    }

    public void closeAccount(){
        int balance = 0;
    }
}
```

Balance is a member variable. It has class scope (can be seen in every method of the class)

```
public class Account{

    int balance = 100;

    public void withdrawFiver(){
        balance = balance -5;
    }

    public void withdrawTenner(){
        int tenner = 10;
        balance = balance – tenner;
    }

    public void withdrawFifty(){
        balance = balance – (tenner * 5);
    }

    public void closeAccount(){
        int balance = 0;
    }
}
```

Tenner (and the second balance variable) are local variables. They have local scope (can only be seen in the method in which they are declared)

Member vs Local Variables

- Member variables can be declared anywhere within the class
  - order between methods and member variables does not matter
  - (order between member variables matter, e.g., initialisation)
- Local variables must be declared before used.

Summary

- Quick Recap:
  - Objects vs. Classes
  - If/else
- Variables: Primitives and Objects
  - How they are Defined
  - How they are Stored
  - How they are Passed
- Introduction to Scope